HANDS-ON PYTHON PARALLEL BENCHMARKING & OPTIMIZATION LAB

Performance Evaluation and Optimization in Parallel Computing

Empirical Scaling, Shared-Memory Contention, Roofline Modeling, and Dynamic Scheduling in Python

Prepared by: Sufyan bin Uzayr

Minimum Runtime: 120 Minutes (5 Sequential Stages)

Mode: Solo Laptop Execution (Offline)

Constraints: No Paper, No Internet, No AI Tools

STRICT EXECUTION PROTOCOL & ZERO-AI VERIFICATION DESIGN

- **Solo, Offline, Pure-Python Execution:** All coding, benchmarking, data tabulation, and written analysis must be executed solely on your local laptop using standard Python 3 (`multiprocessing`, `shared_memory`, `ctypes`, `array`, `time`). Disconnect Wi-Fi/Ethernet before beginning. No scratch paper, external notes, third-party packages, or AI assistants are permitted.
- **Bypassing the GIL for True Hardware Concurrency:** Standard Python `threading` is serialized by the Global Interpreter Lock (GIL). Every parallel kernel in this laboratory must use OS-level worker processes (`multiprocessing.Process`) mapped onto physical CPU cores and POSIX/Windows shared RAM (`multiprocessing.shared_memory`) so your code physically exercises multi-core caches, interconnects, and memory controllers.
- **Hardware-Bound Telemetry & Cross-Task State Chaining:** Each phase interrogates your laptop CPU and emits a deterministic 64-bit cryptographic verification checksum (`VERIFY_HASH`). Subsequent phases require you to feed your measured hardware parameters forward. Static or fabricated numbers will fail mathematical and hardware-topology consistency audits.
- **Submission Artifact:** Submit a single digital report file named `lab_report_<YourID>.md` (or `.txt`) containing your raw terminal telemetry logs, completed tables, Python 3 implementations, and hardware-grounded analytical answers.

Time Allocation & Mandatory Pacing Schedule (120 Minutes Minimum)

Phase	Window	Core Engineering & Experimental Activities	Duration
Phase 0	0:00 – 0:15	Offline setup, CPU/cache topology audit, building the pure-Python `harness.py` module, timer noise-floor calibration.	15 mins
Phase 1	0:15 – 0:40	Shared-memory false sharing stride sweep (1 to 32 64-bit words across $1 \dots P$ processes), cache-line boundary detection, Karp-Flatt $e(p)$.	25 mins
Phase 2	0:40 – 1:05	Strong scaling (Amdahl fit) vs. weak scaling (Gustafson fit) sweep across every integer core count $p = 1 \dots P_{\log}$.	25 mins
Phase 3	1:05 – 1:40	Empirical machine Roofline (DRAM streaming bandwidth + unrolled FMA peak), 3-stage GEMM optimization, 10-point tile (BS) sweep.	35 mins
Phase 4	1:40 – 2:00	Skewed workload scheduling (static block vs. cyclic vs. shared-memory dynamic chunk sweep), load imbalance λ audit, final report.	20 mins

1. Phase 0: Hardware Topology Audit & Harness Calibration (15 Mins)

1.1 System Interrogation (No Internet Required)

Open a terminal on your laptop and execute the native hardware inspection commands for your operating system to record your exact microarchitectural baseline:

```
# Linux:
lscpu | grep -E "Model name|Socket|Core|Thread|L1d|L2|L3" && getconf
    LEVEL1_DCACHE_LINESIZE

# macOS:
sysctl -n machdep.cpu.brand_string && sysctl hw.physicalcpu hw.logicalcpu hw.
    cachelinesize hw.l1dcachesize hw.l2cachesize hw.l3cachesize

# Windows (PowerShell):
Get-CimInstance Win32_Processor | Select Name, NumberOfCores,
    NumberOfLogicalProcessors, L2CacheSize, L3CacheSize
```

Create a directory `parallel_lab/` and initialize `lab_report_<YourID>.md`. Record this hardware baseline table at the top of your report:

Hardware Parameter	Measured Value	Hardware Parameter	Measured Value
CPU Model String	[Fill from OS]	L1 Data Cache (per core)	[Fill in KB]
Physical Cores (P_{phys})	[Fill integer]	L2 Cache Size	[Fill in KB/MB]
Logical Threads ($P_{\log}$)	[Fill integer]	L3 Cache Size (Shared)	[Fill in MB]
Cache Line Size (L_{bytes})	[Fill in Bytes]	Python Version & OS	[e.g., Python 3.11 / Linux]

1.2 The Self-Contained Python Benchmark & Fingerprint Harness

Save the following zero-dependency Python 3 module as `harness.py`. It provides monotonic nanosecond timing via `time.perf_counter_ns()`, warm-up stabilization, statistical median/IQR calculation, and the hardware-bound verification hash generator. Note: on macOS and Windows, `multiprocessing` uses `spawn`, so all worker functions in subsequent tasks **must be defined at module top level** inside an `if __name__ == "__main__":` guard.

Listing 1: harness.py — Zero-Dependency Python 3 Telemetry & Verification Framework

```
from dataclasses import dataclass
import multiprocessing as mp
import time
from typing import Callable

@dataclass
class BenchStats:
    min_ms: float
    median_ms: float
    iqr_ms: float
    max_ms: float

def fnv1a_hash(s: str) -> int:
    h = 14695981039346656037
    for b in s.encode("utf-8"):
        h ^= b
        h = (h * 1099511628211) & 0xFFFFFFFFFFFFFFFF
    return h

def run_benchmark(fn: Callable[[], None], warmup_runs: int = 3, timed_runs: int = 9)
    -> BenchStats:
    for _ in range(warmup_runs):
        fn()
    samples = []
    for _ in range(timed_runs):
```

```
        t0 = time.perf_counter_ns()
        fn()
        t1 = time.perf_counter_ns()
        samples.append((t1 - t0) / 1e6)
    samples.sort()
    mid = timed_runs // 2
    med = samples[mid] if timed_runs % 2 == 1 else 0.5 * (samples[mid - 1] + samples[
        mid])
    q25 = samples[timed_runs // 4]
    q75 = samples[(3 * timed_runs) // 4]
    return BenchStats(samples[0], med, q75 - q25, samples[-1])

def emit_verify_tag(task_id: str, student_seed: int,
                    primary_metric: float, secondary_metric: float) -> str:
    hw = mp.cpu_count()
    payload = (
        f"{task_id}|{student_seed}|{hw}|"
        f"{primary_metric:.4f}|{secondary_metric:.4f}"
    )
    sig = fnv1a_hash(payload)
    return f"VERIFY_HASH[{task_id}]: {sig:016X} (P_log={hw})"
```

Phase 0 Checkpoint: Write and run `task0_cal.py` to query `mp.cpu_count()`, measure average call latency (ns) over 2×10^6 clock reads, and emit `VERIFY_HASH[PHASE0]` with your Student ID as `student_seed`. Paste the console output into your report.

2. Phase 1: Forensic Diagnosis of False Sharing & Cache-Line Contention (25 Mins)

2.1 Experimental Objective

Modern multi-core processors enforce cache coherence (MESI/MOESI) at cache-line granularity (64 bytes on x86-64; 128 bytes on Apple M-series). By mapping a single `multiprocessing.shared_memory` (`SharedMemory`) buffer into multiple OS processes and writing through `ctypes.c_uint64` pointers, distinct CPU cores update adjacent 8-byte words on the exact same physical cache line, triggering hardware invalidation storms. You will empirically discover your CPU's coherence line width and diagnose the overhead using the **Karp-Flatt Metric**.

2.2 Implementation Specification (`task1_falsesharing.py`)

Write a Python 3 program that allocates a shared memory block (`SharedMemory(create=True, size=4096)`) and spawns p worker processes (`mp.Process`) to execute $M = 2.4 \times 10^7$ total deterministic read-modify-write accumulation steps across the processes (M/p iterations per worker).

- Each worker $t \in \{0, \ldots, p-1\}$ attaches to `shm_name`, casts the buffer to a 64-bit unsigned integer array, and updates index `slot_idx = t * stride` in a tight loop:

```
def fs_worker(shm_name: str, slot_idx: int, iters: int) -> None:
    shm = shared_memory.SharedMemory(name=shm_name)
    arr = (ctypes.c_uint64 * 512).from_buffer(shm.buf)
    acc = arr[slot_idx]
    mask = 0xFFFFFFFFFFFFFFFF
    mult = 6364136223846793005
    for i in range(iters):
        acc = (acc * mult + (i | 1)) & mask
        arr[slot_idx] = acc  # Direct hardware store to shared L1 cache line
    del arr                  # Release buffer export before closing SharedMemory
    shm.close()
```

- **Mandatory Empirical Sweep:** Run `run_benchmark(..., warmup_runs=2, timed_runs=7)` across:
  1. **Stride Sweep at Max Physical Cores ($p = P_{\text{phys}}$):** Test `stride` $\in \{1, 2, 4, 8, 16, 32\}$ (corresponding to $8, 16, 32, 64, 128, 256$ bytes of separation between process slots).
  2. **Process Scaling Sweep ($p \in \{1, 2, 4, \ldots, P_{\text{log}}\}$):** Run the full process scaling sweep twice: once with `stride = 1` (severe false sharing) and once with `stride = 16` (128-byte isolation).

2.3 Analytical & Mathematical Calculations Required

1. **Table 1A (Stride Sensitivity):** Record median execution time (ms), IQR (ms), and throughput (Mops/s) for all 6 stride values at $p = P_{\text{phys}}$. Identify the exact byte offset where execution time drops precipitously.
2. **Table 1B (Karp-Flatt Diagnostic Series):** For both `stride = 1` and `stride = 16`, compute Speedup $S_p = T_1/T_p$, Parallel Efficiency $E_p = S_p/p$, and the experimental Karp-Flatt serial fraction:

$$e(p) = \frac{\frac{1}{S_p} - \frac{1}{p}}{1 - \frac{1}{p}} \quad \text{for } p \geq 2$$

3. **Forensic Explanation:** Using your Table 1B values, contrast the trajectory of $e(p)$ as p grows under `stride = 1` versus `stride = 16`. Explain why $e(p)$ rises sharply under `stride = 1` on your CPU. Emit `VERIFY_HASH[PHASE1]` passing `median_ms(stride=1)` and `median_ms(stride=16)` at $p = P_{\text{phys}}$.

3. Phase 2: Strong Scaling, Weak Scaling & Architectural Ceilings (25 Mins)

3.1 Experimental Objective

Empirically separate **Amdahl's Strong Scaling** (fixed problem size, latency-bound) from **Gustafson-Barsis's Weak Scaling** (scaled problem size, capacity-bound) on your laptop, and observe the microarchitectural inflection when worker count crosses from physical cores (P_{phys}) into Simultaneous Multithreading / Hyper-Threading (P_{log}).

3.2 Implementation Specification (`task2_scaling.py`)

Construct a hybrid two-stage pipeline executing per trial over N computational elements:

- **Stage A (Strictly Serial Reduction & Prefix Filter):** Executed in the parent process over the first 5% of elements ($0.05N$), performing 12 transcendental operations (`math.sin`, `math.sqrt`) per element to establish a deterministic serial bottleneck.
- **Stage B (Embarrassingly Parallel Compute Kernel):** Partitioned evenly across a persistent `mp.Pool(processes=p)` (created outside the timed region so process fork/spawn overhead does not pollute kernel scaling!) over all N elements, performing 35 floating-point multiply-add and trigonometric iterations per element with zero shared writes.

1. **Experiment 2A — Strong Scaling Sweep:** Fix $N = 1.6 \times 10^6$ elements. Sweep $p \in \{1, 2, 3, 4, \ldots, P_{\text{log}}\}$ (test *every* integer core count up to P_{log}). Record T_p^{strong} (median ms), $S_p^{\text{strong}} = T_1/T_p^{\text{strong}}$, and E_p^{strong}.

2. **Experiment 2B — Weak Scaling Sweep:** Set base per-core workload $N_0 = 2.5 \times 10^5$ elements. For each $p \in \{1, 2, 3, 4, \ldots, P_{\log}\}$, set total workload $N(p) = p \cdot N_0$ (keeping work per worker constant). Record T_p^{weak} (median ms), Weak Scaling Efficiency $E_p^{\text{weak}} = T_1^{\text{weak}}/T_p^{\text{weak}}$, and Gustafson Scaled Speedup:

$$S_p^{\text{Gustafson}} = p - \alpha(p-1), \quad \text{where } \alpha = \frac{T_{\text{Stage A}}(p)}{T_p^{\text{weak}}}$$

3.3 Analytical Deliverables

1. Measure the exact serial fraction $s = T_{\text{Stage A}}(1)/T_1^{\text{strong}}$ at $p = 1$. Compute Amdahl's theoretical maximum speedup $S_\infty = 1/s$ and theoretical speedup at $p = P_{\log}$. Compare against your observed $S_{P_{\log}}^{\text{strong}}$.
2. Analyze the performance curve across the boundary $p = P_{\text{phys}} \to P_{\text{phys}} + 1$ (if your CPU supports SMT/Hyper-Threading) or across performance/efficiency core clusters (if your CPU is a hybrid P-core/E-core architecture). Quantify the exact percentage drop in marginal speedup per added worker.
3. Emit `VERIFY_HASH[PHASE2]` using $S_{P_{\log}}^{\text{strong}}$ and $S_{P_{\log}}^{\text{Gustafson}}$.

4. Phase 3: Empirical Roofline Modeling & Multi-Level Cache Tiling (35 Mins)

4.1 Part 3A: Characterizing Your Laptop's Hardware Roofline (12 Mins)

Before optimizing a kernel, measure the physical ceilings of your laptop's memory bus and floating-point pipeline in pure Python. Write `task3a_roofline.py` executing two multi-process microbenchmarks across all P_{phys} cores:

1. **Sustained DRAM Bandwidth ($\text{BW}_{\text{sustained}}$ in GB/s):** Allocate two shared-memory byte buffers A and B of size 96 MB each (192 MB total footprint, exceeding your L3 cache!). Measure the median time across P_{phys} workers to execute parallel `memoryview` slice copies (`mv_a[start:end] = mv_b[start:end]`), which invoke native hardware vector `memcpy`. Since each byte copied requires 1 read + 1 write (2 bytes of DRAM traffic), compute:

$$\text{BW}_{\text{sustained}} = \frac{2 \times 96 \times 10^6}{T_{\text{median}} \times 10^9} \text{ GB/s}$$

2. **Sustained Python Compute Ceiling ($P_{\text{sustained}}$ in GFLOP/s):** Execute an L1/register-resident loop across P_{phys} processes performing 8.0×10^7 fused multiply-add FLOPs per worker using 16 unrolled local scalar variables (`x0 = x0 * a + b; ... x15 = x15 * a + b;`) to saturate interpreter dispatch and FP execution ports. Compute $P_{\text{sustained}}$ in GFLOP/s.
3. **Empirical Ridge Point (I^*):** Calculate $I^* = P_{\text{sustained}}/\text{BW}_{\text{sustained}}$ (FLOPs/Byte).

4.2 Part 3B: Progressive GEMM Optimization & Tile-Size Sweep (23 Mins)

Implement three parallel versions of double-precision Dense Matrix Multiplication $C = A \times B$ for square matrices of dimension $N = 384$ stored as flat 1D buffers (`array('d')` / `SharedMemory`, total footprint $= 3 \times 384^2 \times 8$ B $= 3.54$ MB, exceeding L1/L2 caches) across P_{phys} worker processes:

- **Variant 1 (Naive i-j-k Parallel GEMM):** Rows i partitioned across workers; middle loop j; inner loop k computes `dot += A[i * N + k] * B[k * N + j]`, traversing B with stride N (severe L1/L2 cache misses and repeated index arithmetic).

- **Variant 2 (Loop-Reordered i-k-j Parallel GEMM):** Rows i partitioned across workers; middle loop k hoists `a_ik = A[i * N + k]`; inner loop j traverses contiguous row slices of B and C with unit stride (1).
- **Variant 3 (6-Loop Cache-Blocked GEMM with Tile Slice Hoisting):** Partition loops (i, j, k) into blocks of size $BS \times BS$. Inside the micro-kernel, order loops (i, k, j) and hoist `a_ik` outside the innermost j-loop. Verify numerical equivalence against Variant 2 ($\max |C_{ij}^{(3)} - C_{ij}^{(2)}| < 10^{-9}$).

Mandatory 10-Point Tile Size (BS) Sweep: Benchmark Variant 3 across all 10 tile sizes:

$$BS \in \{4, 8, 12, 16, 24, 32, 48, 64, 96, 128\}$$

Record median execution time (ms) and achieved throughput GFLOP/s $= \frac{2N^3}{T_{\text{median}} \times 10^9}$ for Variant 1, Variant 2, and all 10 points of Variant 3.

4.3 Roofline & Cache Capacity Analysis

1. Calculate the working-set footprint $W_{\text{tile}}(BS) = 3 \times BS^2 \times 8$ bytes for each tested BS. Compare the empirical optimal tile size BS^* (where GFLOP/s peaks on your laptop) against your Phase 0 L1d and L2 cache capacities and loop overhead.
2. Compute the analytical operational intensity $I_{\text{naive}} \approx 0.125$ FLOP/Byte versus $I_{\text{tiled}}(BS^*) \approx \frac{2(BS^*)^3}{3(BS^*)^2 \times 8} = \frac{BS^*}{12}$ FLOP/Byte. Map Variant 1, Variant 2, and Variant 3 (BS^*) onto your laptop's Roofline model, calculating the percentage of attainable performance achieved.
3. Emit `VERIFY_HASH[PHASE3]` using $\text{GFLOP/s}_{\text{Variant1}}$ and $\text{GFLOP/s}_{\text{Variant3}}(BS^*)$.

5. Phase 4: Skewed Workloads, Load Imbalance & Dynamic Work-Stealing (20 Mins)

5.1 Experimental Objective

Real-world parallel applications (sparse graphs, adaptive mesh refinement, ray tracing) exhibit severe computational skew across loop iterations. Static partitioning leaves fast cores idling at join barriers. You will quantify load imbalance and tune a shared-memory dynamic work-stealing scheduler in Python.

5.2 Implementation Specification (`task4_scheduling.py`)

Define a triangular skewed workload over $N = 12,000$ outer items, where item $i \in \{0, \ldots, N-1\}$ performs a deterministic loop of $K(i) = 1 + \lfloor (i/60)^2 \rfloor$ heavy trigonometric iterations (`math.sin`, `math.cos`, `math.sqrt`; item 11,999 performs 40,000× more work than item 0!). Execute across $p = P_{\text{phys}}$ worker processes using three scheduling architectures:

1. **Strategy 1 — Contiguous Block Static:** Worker t processes contiguous range $[t \cdot \frac{N}{p}, (t+1)\frac{N}{p})$.
2. **Strategy 2 — Cyclic (Round-Robin) Static:** Worker t processes indices $i = t, t+p, t+2p, \ldots$.
3. **Strategy 3 — Shared-Memory Dynamic Chunk Queue:** Workers claim chunks of size C from a shared atomic/synchronized counter (`mp.Value('i', 0)` or `mp.RawValue` + `mp.Lock`). Sweep chunk sizes $C \in \{1, 4, 16, 64, 256, 1024\}$.

5.3 Load Imbalance Telemetry & Analysis

1. Instrument each worker process t (via a shared `mp.Array('d', p)`) to record its individual active execution time $t_{\text{worker}}[t]$ (in ms) before exiting. For Strategy 1, Strategy 2, and all 6 chunk sizes of

Strategy 3, record total wall-clock time T_{wall} and compute the **Load Imbalance Factor (λ)**:

$$\lambda = \frac{\max_{0 \le t < p} t_{\text{worker}}[t]}{\frac{1}{p}\sum_{t=0}^{p-1} t_{\text{worker}}[t]} \quad \text{(where } \lambda = 1.00 \text{ indicates perfect balance)}$$

2. Explain the trade-off observed in Strategy 3 between $C = 1$ (minimal tail skew, maximum IPC/lock cache-line contention on the shared index) and $C = 1024$ (minimal synchronization overhead, high tail-end load imbalance). Identify C^* for your CPU.
3. Emit `VERIFY_HASH[PHASE4]` using $T_{\text{wall}}(\text{Strategy 1})$ and $T_{\text{wall}}(\text{Strategy 3}, C^*)$.

FINAL DIGITAL SUBMISSION CHECKLIST (`lab_report_<YourID>.md`)

Ensure your single digital file contains all five sections below before time expires:

1. **Hardware Baseline & Verification Hashes:** Complete Phase 0 CPU/cache specification table plus all five verbatim `VERIFY_HASH[PHASE0..PHASE4]` output lines.
2. **Phase 1 Telemetry & Karp-Flatt Analysis:** Tables 1A and 1B, cache-line boundary identification, and Karp-Flatt $e(p)$ diagnostic proof.
3. **Phase 2 Scaling Matrices:** Strong vs. Weak scaling tables across $p = 1 \dots P_{\log}$, Amdahl asymptotic ceiling comparison, and SMT/core-heterogeneity inflection analysis.
4. **Phase 3 Empirical Roofline & GEMM Sweep:** Measured $\text{BW}_{\text{sustained}}$, $P_{\text{sustained}}$, Ridge Point I^*, 10-point BS throughput table, and Roofline efficiency mapping.
5. **Phase 4 Scheduling & Load Imbalance Table:** Wall-clock times and λ imbalance factors across Static, Cyclic, and Dynamic chunk sweeps, plus your complete Python 3 source code appendix.

INSTRUCTOR FORENSIC AUDIT GUIDE (FOR RAPID SUBMISSION VERIFICATION)

- **Checksum Integrity Check:** Recompute `fnv1a_hash` over `task_id|student_id|P_log|metric1|metric2` against the student's reported table values; any post-hoc manual editing of numbers invalidates the 64-bit hex digest.
- **Microarchitectural Consistency Audit:** Verify that (a) the Phase 1 false-sharing inflection occurs at 64 bytes (`stride=8`) on x86-64 or 128 bytes (`stride=16`) on Apple Silicon; (b) Phase 2 exhibits a marginal speedup inflection at $p > P_{\text{phys}}$; and (c) Phase 3 optimal tile footprint $3(BS^*)^2 \times 8$ bytes fits cleanly inside the student's reported L1d/L2 cache capacity.